

影像處理核心技術 與應用實作

TKINTER 互動介面 × NUMPY 幾何投影變換

國立澎湖科技大學

資工二甲 | 林柏翰

整體架構 OVERALL ARCHITECTURE



01. 圖像旋轉縮放工具

探討 Tkinter GUI 狀態管理機制，
以及使用 PIL 進行影像的高效縮放
與旋轉邏輯實作。



02. 圖像疊加工具

分析 NumPy 單應性矩陣
(Homography Matrix) 的數學運
算，與多邊形遮罩合成技術。



03. 技術總結與展望

對比兩種不同影像處理架構（物件
導向 vs 函數式運算）的優勢與適
用場景。

SECTION 01

圖像旋轉縮放工具

基於 Tkinter 與 PIL 的動態影像裝
飾框架

核心架構 | OOP 與狀態機

■類別封裝 (OOP)

使用 ImageDecoratorApp 類別整合 UI 元件與處理邏輯，確保程式碼具備高內聚性與低耦合度。透過物件導向設計有效隔離各項功能模組，避免全域變數汙染，提升系統維護性。

■跨平台資源管理

導入 Python 的 pathlib 模組進行檔案路徑處理，自動適應 Windows、macOS 與 Linux 的路徑分隔符號差異。確保各類素材圖片與圖標在不同環境下均能穩定載入，徹底杜絕因路徑報錯導致的程式崩潰。

■狀態驅動更新

核心採用 `current_angle` 與 `current_scale` 變數實作輕量級狀態機。當使用者透過滑桿或輸入框變更參數時，系統會立即偵測狀態變化並自動觸發畫布重繪，達成流暢的即時預覽效果。

■Lambda 閉包事件

在 GUI 按鈕與控制元件中廣泛運用 `lambda` 匿名函式與閉包技術，實現簡潔的參數傳遞。這使開發者無需為每個相似按鈕定義冗長的回调函式，大幅簡化代碼結構並增強系統擴展靈活性。

實作結果 | 重採樣與定錨邏輯

動態縮放與旋轉展現

如右圖模擬所示，圖片經過 -60° 旋轉與 $0.5x$ 縮放，依然完美定錨於畫布正中央。

- > **保留細節**：縮放採用 LANCZOS，旋轉採用 BICUBIC 以減少邊緣鋸齒。
- > **透明度支援**：轉換為 RGBA 模式，確保擴展 (expand) 後的邊緣保持透明而非黑邊。
- > **絕對置中演算**：透過計算 $(\text{canvas_size} - \text{img_size}) // 2$ 精準處理 Bounding Box 改變的問題。



SECTION 02

圖層疊加工具

運用線性代數實現精確的空間投影

與變換

技術原理 | 單應性矩陣投影

NumPy 矩陣運算管道

拋棄簡易的 2D 縮放，採用真正的 3D 至 2D 平面投影技術，將商標完美貼合於傾斜的建築空地。

- > **數學模型**：建立單應性矩陣 (Homography) 對應 Source 與 Destination 頂點。
- > **最小平方法**：利用 `np.linalg.lstsq` 高效計算出 8 個未知的透視變換係數。

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

```
def find_perspective_coeffs(src_pts, dst_pts):  
    """  
    計算透視變換係數 (從 dst → src 的逆映射)  
    參數: src_pts / dst_pts 為 4 個 (x, y) 點，順序一致  
    回傳: 8 個變換係數 (a,b,c,d,e,f,g,h)  
    """  
    matrix = []  
    for (xs, ys), (xd, yd) in zip(src_pts, dst_pts):  
        matrix.append([xd, yd, 1, 0, 0, 0, -xs*xd, -xs*yd])  
        matrix.append([0, 0, 0, xd, yd, 1, -ys*xd, -ys*yd])  
    A = np.array(matrix, dtype=np.float64)  
    b = []  
    for (xs, ys), _ in zip(src_pts, dst_pts):  
        b += [xs, ys]  
    b = np.array(b, dtype=np.float64)  
  
    coeffs, _, _, _ = np.linalg.lstsq(A, b, rcond=None)  
    return tuple(coeffs)
```


輸入素材 | BACKGROUND & LOGO



目標背景建築空地



覆蓋商標

完美合成 | 透視貼圖成果

精確的邊界控制

解決了直接貼圖產生不規則黑邊的痛點，透過遮罩技術進行裁切與融合：

- **L 模式遮罩生成**：建立與背景同寬高的 8-bit 灰階遮罩。
- **多邊形填充 (Polygon)**：使用 ImageDraw.polygon 依據四角座標填入純白，精確定義有效貼合區域。
- **預覽診斷模式**：內建 Preview Flag，開啟後自動於四角繪製圓圈，大幅降低開發者微調點位時間。



核心技術總整 | SUMMARY TABLE

專案項目	技術焦點	數學與邏輯複雜度	架構特色
圖像旋轉縮放工具	Tkinter UI / PIL 基礎變換	★☆☆☆☆ (線性仿射)	OOP 類別封裝、狀態機設計
圖層疊加工具	NumPy 矩陣 / 單應性投影	★★★★☆ (投影幾何方程)	Functional 流程、資料驅動
共通核心優勢	跨平台相容 / 記憶體控制	-	高度模組化、無需龐大外部引擎

結論與展望 | FUTURE OUTLOOK

從底層運算到視覺化

透過本次專案，我們展示了如何結合 Python 的科學計算能力 (NumPy) 與創意影像繪圖 (Pillow)，不依賴龐大的 OpenCV 也能實現商業等級的 AR 貼圖效果。

未來優化方向：

01 導入即時拖拽 (Drag & Drop) 來動態調整四個透視點。

02 加入 OpenCV 進行特徵點自動辨識，實現無需手動輸入座標的自動化合成。

THANK YOU FOR LISTENING

感謝您的聆聽與指教

QUESTIONS & ANSWERS